

**UNIVERSIDADE FEDERAL DO RIO DE
JANEIRO**

DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

**RELATÓRIO DE IMPLEMENTAÇÃO
E.D.: CLASSE PILHA**

Integrantes e DREs:

Gabriel de Souza Machado González – DRE: 120066108

Mateus Fernandes Santos – DRE: 123224894

Ramon Ramirez – DRE: _____

Zico Costa – DRE: 124011595

Repositório Git:

<https://github.com/mateus-ma/classe-pilha.git>

Rio de Janeiro

2026

1 Análise da Solução Gerada por Inteligência Artificial

A IA resolveu bem os problemas, apesar de ter apresentado soluções excessivamente técnicas para o contexto do trabalho. Após estudar o código, não encontramos pontos que necessitassem de refatoração, o que faz sentido, pois o problema é bem simples.

Não houve erros de execução, mas o código gerado inicialmente não condizia com a estrutura de repositório sugerida pela própria IA para a implementação em C++, que contava com um arquivo a mais. Um prompt extra resolveu essa questão.

2 Análise de Trecho Específico em Python

Um trecho da implementação em Python que chamou atenção foi:

```
1 def empilha(self, dado):
2     if self.pilha_esta_cheia():
3         raise PilhaCheiaErro("A pilha esta cheia. Nao e possivel
4         empilhar mais elementos.")
5
6     try:
7         self._dados.append(dado)
8     except (TypeError, ValueError) as err:
9         raise TipoErro(f"O dado '{dado}' nao e compativel com o
10        tipo estipulado '{self._typecode}'.") from err
```

A estrutura `array.array` já exige que todos os dados sejam do mesmo tipo e levanta um `ValueError` ou `TypeError` caso um tipo diferente seja passado. Como nas especificações da tarefa foi exigido que o erro exibido seja o customizado `TipoErro`, a IA foi capaz de perceber e contornar isso.

3 Resultados dos Testes

Os testes também foram gerados por IA. Todos os métodos de cada classe foram testados, é possível verificar isso pelo “OK!” em C++/JS e os 7 pontos em Python. Repare bem a diferença entre os tempos de execução, aqui, C++ e JavaScript apresentam performances bem semelhantes.

C++

Executando testes unitarios (C++)... OK!

[C++ Estresse - 10000000 elementos]

Tempo Empilhar: 0.0392435s

Tempo Desempilhar: 0.0179506s

JavaScript

Executando testes unitários (JavaScript)...

OK!

```
[JS Estresse - 10000000 elementos]
Tempo Empilhar: 0.0385s
Tempo Desempilhar: 0.0190s
```

Python

```
Tempo de Empilhar: 0.1707s
Tempo de Desempilhar: 0.1288s
.....
```

```
Ran 7 tests in 0.300s
```

```
OK
```

Lista de testes em Python:

- Operações básicas;
- Teste de exceção para pilha cheia;
- Teste de exceção para pilha vazia;
- Teste de exceção para tipo incompatível;
- Teste do método troca;
- Teste de troca quando não há elementos suficientes;
- Teste de estresse com 1.000.000 de elementos.

4 Conclusão

Em suma, a implementação da estrutura de dados Pilha em C++, JavaScript e Python cumpriu com sucesso todos os requisitos propostos. A análise do código gerado pela IA permitiu compreender nuances importantes de implementação: como o tratamento do `array.array` em Python via bloco `try-except`. Além disso, pudemos validar a eficácia dos testes unitários, que cobriram todas as nossas demandas. Por fim, os testes de estresse evidenciaram com clareza o comportamento de cada ambiente de execução, destacando a equivalência e superioridade de performance do C++ e do JavaScript frente ao Python para volumes massivos de operações.